

APRENDIZADO PROFUNDO II

DESENVOLVIMENTO DE UM SMALL LANGUAGE MODEL (SLM)

VISÃO GERAL

Neste trabalho, cada grupo irá projetar, treinar e avaliar um **Small Language Model (SLM)** (Sugere-se que o modelo tenha entre **50 milhões e 200 milhões de parâmetros treináveis**). O objetivo é reproduzir, em escala reduzida, o pipeline completo de desenvolvimento de LLMs: desde o design da arquitetura e o pré-treino em larga escala, passando pelo *mid-training* e pelo *supervised fine-tuning* (SFT), até a avaliação quantitativa e uma demonstração em modo chat.

O trabalho é deliberadamente **aberto**. Não há uma única solução correta: diferentes escolhas de arquitetura, dados e hiperparâmetros levarão a modelos com características distintas. Explorar e justificar essas escolhas é parte central da avaliação.

O desenvolvimento do SLM segue quatro etapas sequenciais:

Pré-treino → Mid-Training → SFT → Avaliação & Demo

A entrega intermediária cobre o pré-treino. A entrega final cobre todas as etapas.

ETAPA 1 — PRÉ-TREINO

Objetivo

O pré-treino tem como objetivo aprender representações gerais de linguagem a partir de um grande corpus de texto não rotulado. O modelo deve aprender a prever o próximo *token* de forma autorregressiva (*next-token prediction*).

Projeto da Arquitetura

Cada grupo deve projetar a arquitetura do seu SLM. O ponto de partida é um **transformer decoder-only** (estilo GPT), mas os grupos estão **encorajados a explorar e justificar variantes**. A seguir estão os principais componentes a considerar:

Componente	Exemplos e Alternativas a Explorar
Positional Encoding	RoPE , ALiBi, aprendido, sinusoidal
Mecanismo de Atenção	MHA , GQA (<i>Grouped-Query Attention</i>), MQA, Flash Attention
Função de Ativação	GELU , SwiGLU, ReLU, SiLU
Normalização	LayerNorm (pre-norm), RMSNorm, post-norm
Otimizador	AdamW , Muon, Adan; schedule: cosseno, WSD
Tokenizer	BPE (GPT-2) , BPE com vocab maior, SentencePiece, treinado do zero

A contagem de parâmetros deve se encaixar no intervalo **50M – 200M parâmetros treináveis**. Grupos que extrapolarem esse intervalo deverão justificar tecnicamente a decisão.

Dataset e Scaling Laws

A escolha do corpus de pré-treino e da quantidade de tokens é responsabilidade do grupo. Recomenda-se o uso das **Chinchilla Scaling Laws** como guia: o número

ótimo de tokens de treinamento é aproximadamente $20 \times N$, onde N é o número de parâmetros. Por exemplo, um modelo de ~63M parâmetros deve ser treinado em pelo menos ~1,3B tokens (nesses casos normalmente arredondamos para 2B).

Sugestões de datasets de alta qualidade para pré-treino:

- **FineWeb-Edu** (HuggingFaceFW/fineweb-edu) — texto web filtrado por qualidade educacional
- **The Pile** — corpus diversificado de alta qualidade
- **RedPajama** — reprodução open-source do LLaMA training mix
- **SlimPajama** — versão deduplicada do RedPajama

Infraestrutura e Tempo de Treinamento

O servidor do curso está disponível para o treinamento. Detalhes sobre acesso constam no moodle.

Sugere-se:

- Usar treinamento distribuído (DDP com 2 GPUs) para o pré-treino.
- Monitorar a loss de treino ao longo do tempo e incluir o gráfico na apresentação.
- Salvar checkpoints regularmente para garantir a continuidade do treinamento em caso de falha.

REFERÊNCIA DE TEMPO O pré-treino de um modelo de ~63M parâmetros treinado em ~2B tokens utilizando 2 GPUs do servidor tipicamente leva **em torno de 6 horas**. Planeje o cronograma do grupo levando em conta filas de espera e eventuais interrupções.

Entrega Intermediária

ENTREGA 1 — PRÉ-TREINO PRAZO DISPONÍVEL NO MOODLE Entregar via repositório GitHub (link no Moodle) até a data definida, contendo:

1. **Repositório GitHub** com código de pré-treino organizado e documentado (README com instruções de execução).
2. **Checkpoint do modelo** (ao menos um checkpoint após treinamento parcial ou completo), hospedado no HuggingFace Hub ou Google Drive.
3. **Gráfico da curva de loss** de treino ao longo dos steps/tokens.
4. **Demonstração de geração de texto** (alguns exemplos de completamento de prompt com o modelo pré-treinado, mesmo que ainda com qualidade baixa).

ETAPA 2 — MID-TRAINING

Objetivo

O mid-training tem como objetivo **especializar o modelo** a partir do checkpoint de pré-treino, expondo-o a dados de natureza conversacional e de raciocínio. Essa etapa utiliza pesos inicializados do pré-treino e continua o treinamento com novos dados e com uma taxa de aprendizado que pode ser diferente em relação ao pré-treino.

Dados

Recomenda-se um mix de dados conversacionais e de raciocínio. Sugestões:

- **SmolTalk** — dataset conversacional de alta qualidade (HuggingFaceTB/smoltalk)
- **GSM8K** — problemas de matemática elementar para capacidade de raciocínio
- **OpenHermes**, **UltraChat** ou outros datasets de instruções de alta qualidade

O grupo pode escolher um mix diferente, desde que **justifique a escolha** (por que esses dados tornam o modelo mais capaz para as etapas seguintes?).

Configuração

Orientações gerais para o mid-training:

- Inicializar a partir do checkpoint final do pré-treino (somente pesos do modelo, sem estado do otimizador).
- Usar uma **taxa de aprendizado menor** do que o pré-treino (típico: $1e-4$ a $3e-4$ com warmup curto).
- Treinar por aproximadamente 1 epoch sobre o dataset (ou um número de tokens equivalente a $\sim 100M-300M$ tokens).
- Formato dos dados: converter todas as amostras para um template de chat (ex: User: ... \nAssistant: ...) e aplicar sequence packing.

ETAPA 3 — SUPERVISED FINE-TUNING (SFT)

Objetivo

O SFT tem como objetivo **alinhar o modelo ao formato instrucional**, ensinando-o a responder perguntas de forma útil e coerente. A principal diferença em relação ao mid-training é o uso de uma **loss mask**: somente os tokens das respostas do assistente contribuem para o cálculo da loss; tokens do prompt (User:) são mascarados com -100.

Dados e Configuração

Para o SFT, recomenda-se:

- Usar o mesmo dataset conversacional do mid-training (ex: SmolTalk) ou um subset curado de alta qualidade.

- Taxa de aprendizado ainda mais baixa (típico: $1e-5$ a $3e-5$) e warmup curto.
- Treinar por 1–2 epochs sobre o dataset de SFT, monitorando o risco de overfitting.
- Inicializar a partir do checkpoint final do mid-training.

ETAPA 4 — AVALIAÇÃO

Métricas Obrigatórias

Cada grupo deve implementar e reportar, no mínimo, **dois tipos de avaliação**:

a) Perplexidade

Calcule a **perplexidade** (PPL) do modelo sobre um conjunto de validação. A PPL é definida como $\exp(\bar{\ell})$, onde $\bar{\ell}$ é a cross-entropy média sobre os tokens, e mede quão bem o modelo prevê os tokens. Valores menores indicam melhor modelo.

A avaliação de perplexidade deve ser feita sobre um split de validação do dataset de pré-treino (textos que o modelo não viu durante o treino) e reportada ao longo do treinamento (curva de validação).

b) Benchmark de Múltipla Escolha

Além da perplexidade, cada grupo deve implementar e reportar os resultados em **pelo menos um** dos benchmarks abaixo:

- **HellaSwag** — completamento de sentenças (raciocínio de senso comum)
- **ARC-Easy / ARC-Challenge** — perguntas de ciências (ensino fundamental/médio)
- **PIQA** — raciocínio físico de senso comum
- **WinoGrande** — resolução de anafóricos ambíguos

Para benchmarks de múltipla escolha, utilize a avaliação por **log-likelihood**: para cada alternativa, calcule a probabilidade do modelo gerar aquele texto dado o contexto, e escolha a alternativa de maior log-verossimilhança. Isso evita a dependência de geração autorregressiva.

Avaliações Adicionais (Opcional)

Grupos que quiserem se destacar podem implementar avaliações adicionais, como:

- Comparação qualitativa de respostas antes e depois do SFT para as mesmas perguntas.
- Avaliação em GSM8K (acurácia em problemas de matemática elementar).
- Avaliação humana: o grupo prepara um conjunto de perguntas e membros externos avaliam qual modelo responde melhor.

DEMO E APRESENTAÇÃO

Demo em Modo Chat

Cada grupo deve desenvolver uma **interface de chat** que permita interagir com o SLM treinado. Uma aplicação **Streamlit** é suficiente. A demo deve:

- Carregar o checkpoint final do SFT.
- Permitir enviar mensagens ao modelo e visualizar as respostas.
- Expor parâmetros de geração ajustáveis (temperatura, top-k, número máximo de tokens).
- Ser executada ao vivo durante a apresentação da disciplina.

Apresentação

Cada grupo terá **10–15 minutos** de apresentação, incluindo a demo ao vivo. A apresentação deve cobrir:

1. Descrição da arquitetura escolhida e justificativa das decisões de design.
2. Análise das Scaling Laws aplicadas: como o número de tokens foi determinado?
3. Curvas de loss de treino (pré-treino, mid-training e SFT).
4. Resultados da avaliação: perplexidade e benchmark(s) implementado(s).
5. Discussão: o que funcionou bem? O que mudaria com mais tempo/recursos?
6. Demo ao vivo do modelo em modo chat.

ENTREGA FINAL

ENTREGA FINAL | **PRAZO DISPONÍVEL NO MOODLE** Todos os itens abaixo devem ser entregues via repositório GitHub (link no Moodle) até a data definida, contendo:

1. **Repositório GitHub organizado** com código de pré-treino, mid-training, SFT, avaliação e demo. README com instruções para reproduzir cada etapa.
2. **Checkpoints**: checkpoint final do SFT (obrigatório), checkpoint do mid-training (recomendado) e do pré-treino (recomendado), hospedados no HuggingFace Hub ou Google Drive.
3. **Resultados de avaliação**: perplexidade no split de validação e resultados no(s) benchmark(s) implementado(s).
4. **Slides da apresentação** (PDF ou link para o Google Slides).
5. **Interface de chat funcional** (código no repositório + instruções para execução local).

Bons diferenciais (podem elevar a nota): explorar arquiteturas alternativas além do GPT padrão (ex: GQA, SwiGLU, RMSNorm, Mamba), usar múltiplos benchmarks, comparar modelos em diferentes estágios (pós pré-treino vs. pós SFT), ou justificar quantitativamente as escolhas de hiperparâmetros via experimentação.

DICAS

Boas Práticas

- **Comece pequeno:** valide o pipeline completo (treino, checkpoint, avaliação, demo) com um modelo menor antes de escalar.
- **Smoke test:** verifique que o modelo consegue overfittar um mini-batch, isso confirma que o pipeline de treino está correto.
- **Checkpoints frequentes:** treinamentos longos no servidor podem ser interrompidos; retomadas devem ser possíveis.
- **Monitore a loss em tempo real:** uma loss que não decresce no início geralmente indica bug no pipeline de dados ou na inicialização.
- **Scripts separados:** separe o código de cada etapa (pré-treino, mid-training, SFT, avaliação) em scripts distintos, cada um com sua configuração própria.

Referências

- **Attention Is All You Need** — Vaswani et al. (2017)
- **Language Models are Few-Shot Learners (GPT-3)** — Brown et al. (2020)
- **Training Compute-Optimal Large Language Models (Chinchilla)** — Hoffmann et al. (2022)
- **LLaMA: Open and Efficient Foundation Language Models** — Touvron et al. (2023)
- **RoFormer: Enhanced Transformer with Rotary Position Embedding** — Su et al. (2022)
- **SmolLM** — HuggingFace (2024): referência de SLM com pipeline similar a este trabalho
- **nanoGPT** — Andrej Karpathy: implementação de referência simples e bem documentada (<https://github.com/karpathy/nanoGPT>)

REGRAS E CONSIDERAÇÕES

- **Grupos:** 3 a 5 integrantes.
- **Uso de código externo:** é permitido usar bibliotecas open-source (PyTorch, HuggingFace Datasets, Tiktoken etc.) e se inspirar em implementações de referência (ex: nanoGPT). Entretanto, o código do pipeline de treino, avaliação e demo deve ser escrito pelo grupo. Qualquer código adaptado de terceiros deve ser explicitamente creditado no repositório.
- **Prazo:** entregas fora do prazo terão desconto de 10% por dia de atraso, até o limite de 50%. Não serão aceitas entregas com mais do que 5 dias de atraso.
- **Apresentação:** não é obrigatório que todos os integrantes participem da apresentação.